EXP.NO:	7	PRODUCER-CONSUMER PROBLEM USING SEMAPHORES
DATE:		

AIM

To implement the Producer-Consumer synchronization problem using semaphores in C, ensuring mutual exclusion and proper buffer management.

PROCEDURE

1. Initialize semaphores: mutex=1, full=0, empty=buffer_size.
2. Producer waits if buffer is full; otherwise produces and signals.
3. Consumer waits if buffer is empty; otherwise consumes and signals.
4. Use menu-driven approach to simulate producer and consumer actions.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1, full = 0, empty = 5, x = 0;

int wait(int s)  { return --s; }
int signal(int s) { return ++s; }

void producer() {
    mutex = wait(mutex);
    empty = wait(empty);
    full  = signal(full);
    x++;
    printf("\nProducer produces item %d", x);
    mutex = signal(mutex);
}

void consumer() {
    mutex = wait(mutex);
    full  = wait(full);
    empty = signal(empty);
    printf("\nConsumer consumes item %d", x);
    x--;
    mutex = signal(mutex);
}

int main() {
    int n;
    printf("\n----- Producer Consumer Problem -----");
    printf("\n1. Producer  2. Consumer  3. Exit");
    while(1) {
        printf("\nEnter your choice: "); scanf("%d",&n);
        switch(n) {
            case 1: if(mutex==1&&empty!=0) producer();
                    else printf("\nBuffer is full!"); break;
            case 2: if(mutex==1&&full!=0) consumer();
                    else printf("\nBuffer is empty!"); break;
            case 3: printf("\nFinal items in buffer: %d\n",x); exit(0);
            default: printf("\nInvalid choice.");
        }
    }
}
```

OUTPUT

```
$ gcc producer_consumer.c
$ ./a.out

----- Producer Consumer Problem -----
1. Producer  2. Consumer  3. Exit
Enter your choice: 1
Producer produces item 1
```

```
Enter your choice: 1
Producer produces item 2
Enter your choice: 1
Producer produces item 3
Enter your choice: 2
Consumer consumes item 3
Enter your choice: 2
Consumer consumes item 2
Enter your choice: 2
Consumer consumes item 1
Enter your choice: 2
Buffer is empty!
Enter your choice: 3
Final items in buffer: 0
```

RESULT

The Producer-Consumer problem was successfully implemented using semaphores. Mutual exclusion was maintained and buffer overflow/underflow conditions were correctly handled.